

Rockchip_Developer_Guide_Android_EQ_DRC_CN

文件标识: RK-KF-YF-774

发布版本: V1.1.0

日期: 2024-06-19

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要介绍Rockchip Android EQ/DRC模块，开发或技术支持人员通过阅读此文档，对Android平台EQ/DRC模块有一个初步的了解，帮助读者开发和调试。

产品版本

芯片名称	内核版本
适配所有芯片	适配所有版本

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	蓝斌元	2023-10-24	初始版本
V1.1.0	张贵朋	2024-06-19	框架调整

目录

Rockchip_Developer_Guide_Android_EQ_DRC_CN

1. 编译

- 1.1 确认项目 BoardConfig.mk 位置
- 1.2 添加编译开关

2. 启用

- 2.1 声明音效
- 2.2 启用音效
 - 2.2.1 动态绑定
 - 2.2.2 Stream HAL 绑定
- 2.3 验证
 - 2.3.1 确认音效注册状态
 - 2.3.2 确认音效绑定状态

3. 参数

- 3.1 参数文件及目录
 - 3.1.1 参数文件
 - 3.1.2 参数目录
- 3.2 参数说明
- 3.3 参数调试
 - 3.3.1 自定义 App 调参
 - 3.3.2 PC 工具调参

1. 编译

1.1 确认项目 BoardConfig.mk 位置

BoardConfig.mk 的位置与芯片型号和 lunch 时选择的 product 有关

比如芯片名称为 `rk3588`, product 为 `rk3588_u`

```
# echo $TARGET_PRODUCT
# rk3588_u
```

则 BoardConfig.mk 的位置为 `device/rockchip/rk3588/rk3588_u/BoardConfig.mk`

1.2 添加编译开关

在项目的 BoardConfig.mk 添加下面的编译开关, 确保 `hardware/rockchip/audio/eqdrc` 被编译

```
BOARD_USE_AUDIO_EQDRC := true
```

2. 启用

2.1 声明音效

在 `frameworks/av/media/libeffects/data/audio_effects.xml` 中声明 EQ/DRC 音效

```
<libraries>
  <!-- 保留已有内容, 添加下面的配置项 -->
  <library name="rkeqdr" path="librkeqdr.so"/>
</libraries>
<effects>
  <!-- 保留已有内容, 添加下面的配置项 -->
  <effect name="eqdrc" library="rkeqdr" uuid="79fe72b2-4182-44c1-b2ea-0002a5d5c51b"/>
</effects>
```

2.2 启用音效

根据应用音效的范围选择绑定方式

2.2.1 动态绑定

在 MediaPlayer 或 AudioTrack 创建时动态绑定 EQ/DRC, 这种方法对局部的音频流有效

```
import android.media.MediaPlayer;
import android.media.audiofx.RockchipEqualizer;

MediaPlayer mediaPlayer = new MediaPlayer();
RockchipEqualizer equalizer = new RockchipEqualizer(0,
mediaPlayer.getAudioSessionId());
equalizer.setEnabled(true);
```

2.2.2 Stream HAL 绑定

在 `audio_effects.xml` 中指定对应的流类型来绑定, 这种方法对全局的特定类型的音频流有效

比如绑定到 `music` 类型的 output stream

```

<!-- 如果没有声明 postprocess，则直接添加下面的配置，忽略下面的注释 -->
<postprocess>
  <!-- 保留已有内容，添加下面的配置项 -->
  <stream type="music">
    <apply effect="eqdrc"/>
  </stream>
</postprocess>

```

也可以绑定多个类型的 output stream

```

<!-- 如果没有声明 postprocess，则直接添加下面的配置，忽略下面的注释 -->
<postprocess>
  <!-- 保留已有内容，添加下面的配置项 -->
  <stream type="music">
    <apply effect="eqdrc"/>
  </stream>
  <stream type="voice_call">
    <apply effect="eqdrc"/>
  </stream>
</postprocess>

```

目前总共有下面几种类型的 output stream:

```

/** All output stream types which support effects.
 * This need to be kept in sync with the xsd streamOutputType.
 */
template <>
constexpr std::pair<audio_stream_type_t, const char*>
STREAM_NAME_MAP<audio_stream_type_t>[] = {
    {AUDIO_STREAM_VOICE_CALL, "voice_call"},
    {AUDIO_STREAM_SYSTEM, "system"},
    {AUDIO_STREAM_RING, "ring"},
    {AUDIO_STREAM_MUSIC, "music"},
    {AUDIO_STREAM_ALARM, "alarm"},
    {AUDIO_STREAM_NOTIFICATION, "notification"},
    {AUDIO_STREAM_BLUETOOTH_SCO, "bluetooth_sco"},
    {AUDIO_STREAM_ENFORCED_AUDIBLE, "enforced_audible"},
    {AUDIO_STREAM_DTMF, "dtmf"},
    {AUDIO_STREAM_TTS, "tts"},
    {AUDIO_STREAM_ASSISTANT, "assistant"},
};

```

2.3 验证

2.3.1 确认音效注册状态

确认声明的音效是否已经注册到 AudioFlinger

通过 `dumpsys media.audio_flinger` 确认

```
# dumpsys media.audio_flinger
[...]
Libraries loaded:
  path: /vendor/lib/soundfx/librkeqsrc.so
  EQ DRC / Rockchip Electronics Co. Ltd.
  UUID: 79fe72b2-4182-44c1-b2ea-0002a5d5c51b
  TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
  apiVersion: 00020000
  flags: 00820204
[...]
```

2.3.2 确认音效绑定状态

确认需要的音效是否能正常绑定到对应的 output stream

在 Android App 播放音频, 通过 `dumpsys media.audio_flinger` 确认

```
# dumpsys media.audio_flinger
[...]
Output thread 0xb40000784395ba60, name AudioOut_D, tid 642, type 0 (MIXER):
  I/O handle: 13
  Standby: no
  [...]
  1 Effect Chains
    1 effects for session 49
      [...]
      Effect ID 11:
        Session State Registered Enabled Suspended:
        00049 003 y y n
        Descriptor:
        - UUID: 79fe72b2-4182-44c1-b2ea-0002a5d5c51b
        - TYPE: 34805d32-2e6d-4d1e-9296-0002a5d5c51b
        - apiVersion: 00000000
        [...]
        - name: EQ DRC
        - implementor: Rockchip Electronics Co. Ltd.
        [...]
      Effect ID 11 HAL dump:
    [...]
  [...]
[...]
```

3. 参数

3.1 参数文件及目录

3.1.1 参数文件

默认参数文件保存在以下两个 bin 文件

```
rkaudio_effect_eqdrc_44100hz_2ch.bin
rkaudio_effect_eqdrc_48000hz_2ch.bin
```

也支持其他预设的参数文件

```
rkaudio_effect_eqdrc_48000hz_2ch_classroom.bin
rkaudio_effect_eqdrc_48000hz_2ch_meeting.bin
rkaudio_effect_eqdrc_48000hz_2ch_standard.bin
```

3.1.2 参数目录

SDK 默认保存到 `hardware/rockchip/audio/eqdrc/configs/`

出厂默认 install 到设备的 `/vendor/etc/`

userdebug 版本下, PC 工具 push 的参数文件会保存到设备的 `/vendor/etc/`

App 使用默认配置时 (preset 为 0), 下发的参数将会保存到设备的 `/data/vendor/audio/`, EQ/DRC 创建时首先加载这个目录的参数文件, 如果没有则加载默认目录的参数文件

3.2 参数说明

请参考 [Rockchip Developer Audio Effect Tool.pdf](#)

3.3 参数调试

支持 App 调参和 PC 工具调参, 共两种调参方式, 根据需要选择即可

3.3.1 自定义 App 调参

如果你在 `audio_effects.xml` 中绑定了 EQ/DRC, 它将作为一个音效处理的实例。此时 App 再绑定的 EQ/DRC 将作为调整参数的通道

```
import android.media.MediaPlayer;
import android.media.audiofx.RockchipEqualizer;

MediaPlayer mediaPlayer = new MediaPlayer();
// 绑定
RockchipEqualizer equalizer = new RockchipEqualizer(0,
mediaPlayer.getAudioSessionId());
// 使能
equalizer.setEnabled(true);
```

调用 `android.media.audiofx.RockchipEqualizer.Settings` 对应的 API 即可, 下面是设置 EQ 参数的例子

```
// 获取所有配置
RockchipEqualizer.Settings settings = equalizer.getProperties();
// 使能 EQ10
settings.setEq10Enabled(1);

// 获取当前 EQ, 总共 2 channel 的 EQ
Collection<RockchipEqualizer.Settings.Eq> eqs = settings.getEqs();
// 创建 EQ 迭代器
Iterator<RockchipEqualizer.Settings.Eq> eqsIterator = eqs.iterator();
// 遍历所有 channel 的 EQ
while (eqsIterator.hasNext()) {
    RockchipEqualizer.Settings.Eq eq = eqsIterator.next();
    Log.d(TAG, "eq: " + eq.toString());

    // 获取当前 channel 的 EQ 的所有 band 配置
    Collection<RockchipEqualizer.Settings.Eq.EqBand> eqBands = eq.getBands();
```

```

// 创建 band 迭代器
Iterator<RockchipEqualizer.Settings.Eq.EqBand> eqBandsIterator =
eqBands.iterator();
// 遍历当前 channel 的 EQ 的所有 band 配置
while (eqBandsIterator.hasNext()) {
    RockchipEqualizer.Settings.Eq.EqBand eqBand = eqBandsIterator.next();
    Log.d(TAG, "band: " + eqBand.toString());
    // 设置 band 参数，具体可以看对应的 set API
    // eqBand.setBoost(0); 滤波器中心频率增益，单位 dB
    // eqBand.setFilter(0); 0 表示 Parametric, 1 表示 LowShelf, 2 表示
HighShelf, 3 表示 GeneralLP, 4 表示 GeneralHP
    // eqBand.setFc(0); 滤波器的中心频率，单位 Hz
    // eqBand.setQ(0); filter 为 Parametric 时，Q 为品质因数，范围为[0.01, 16];
filter 为 LowShelf/HighShelf时，Q 为Slope，范围为[0.01, 2]
    // eqBand.setEnabled(0); 是否启用该滤波器，1 表示启用，0 表示bypass
}
}

// 更新配置
equalizer.setProperties(settings);

```

也可以直接加载预设的配置, 默认不指定使用的是 "custom", 只有在此预设下才可以调整上述的参数

```

mEqualizer.usePreset(0); // 使用 "custom" 预设
mEqualizer.usePreset(1); // 使用 "standard" 预设
mEqualizer.usePreset(2); // 使用 "meeting" 预设
mEqualizer.usePreset(3); // 使用 "classical" 预设
mEqualizer.usePreset(4); // 使用 "classroom" 预设

```

3.3.2 PC 工具调参

PC 工具调参需要确保 build 版本是 userdebug, 确保 PC 可以通过 adb 连接到设备

1) 工具目录

PC 工具默认放置在 SDK 的 `RKTools/windows/AudioEffect_Tool.rar`

2) 工具补充介绍

- `load`: 加载任意目录的参数文件
- `save`: 保存当前参数文件到任意目录
- `push`: 通过 adb push 把当前参数文件保存到设备的 `/vendor/etc/`
- `pull`: 通过 adb pull 保存当前设备的参数文件到所选目录

更多工具的介绍请参考 [Rockchip_Developer Audio Effect Tool.pdf](#)

注: 在 push 和 pull 操作下, 对应参数文件名的格式是 `rkaudio_effect_eqdrc_xhz_ych.bin`, x 和 y 分别是工具设置的采样率和通道数

3) 调参步骤

- 调参前需要在 PC 端执行下面两个指令

```

adb root
adb remount
adb shell setenforce 0

```

- 打开 AudioEffect_tool.exe
- `SampleRate` 采样率需要和原有 pcm_config 的 rate 一致

`hardware/rockchip/audio/tinyalsa_hal/`

```
static struct pcm_config pcm_config = {
    // ...
    .rate = 48000,
    // ...
};
```

- `NumChan` 通道数需要和原有 pcm_config 的 channels 一致

`hardware/rockchip/audio/tinyalsa_hal/`

```
static struct pcm_config pcm_config = {
    .channels = 2,
    // ...
};
```

- 点击 `pull`, 保存设备当前参数文件到所选目录 (此步骤可选)
- 点击 `load`, 选择参数文件所在的目录, 加载参数文件
- 在 Android App 播放音频
- 调整参数后点击 `push` 更新参数文件到设备

此时在设备端通过 logcat 可以查看到重新加载参数文件的日志

```
# logcat | grep eqdrc
06-19 07:33:12.286 714 2068 D eqdrc : eqdrc_fx_process : reload profile
from tuner
```

- 人耳朵听音或使用 AP 等音频测试设备, 确认效果。效果达到预期则进行以下步骤, 否则重复调整参数
- 点击 `save` 保存到 SDK 的 `hardware/rockchip/audio/eqdrc/configs` 目录下